LAPORAN PRAKTIKUM STRUKTUR DATA

Queue

Oleh:

Naufal Arkan Octyandi

2511533005

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU: DR. WAHYUDI, S.T, M.T

ASISTEN LABOR: SHERLY SUKMADIRA

FAKULTAS TEKNOLOGI INFORMASI

PEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 27 APRIL 2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan laporan praktikum Struktur Data yang berjudul “Queue” dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum serta untuk menambah wawasan dan pemahaman mengenai konsep struktur data, khususnya penggunaan Queue. Dalam praktikum ini, penulis mempelajari bagaimana cara menyimpan dan mengelola data dengan prinsip FIFO (First In First Out), serta melakukan operasi dasar seperti enqueue, dequeue, dan pengecekan elemen menggunakan Queue dalam bahasa pemrograman Java.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat banyak kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat bagi pembaca dan dapat menjadi referensi tambahan dalam mempelajari struktur data, khususnya konsep Queue.

Padang, 2026

Tim Penyusun

Naufal Arkan Octyandi

DAFTAR ISI

KATA PENGANTAR ii
DAFTAR ISI iii
BAB I PENDAHULUAN 1
1.1 LATAR BELAKANG 1
1.2 TUJUAN PRAKTIKUM 1
1.3 MANFAAT PRAKTIKUM 1
BAB II PEMBAHASAN PRAKTIKUM 2
2.1 Implementasi Queue Menggunakan Array 2
2.2 Pengujian Queue (Class Driver) 3
2.3 Reverse Data pada Queue Menggunakan Stack 4
2.4 Iterasi Queue Menggunakan Iterator 4
2.5 Implementasi Queue Menggunakan LinkedList 5
BAB III KESIMPULAN 7
3.1 KESIMPULAN 7
DAFTAR PUSTAKA 8

BAB I
PENDAHULUAN

1.1 LATAR BELAKANG

Dalam perkembangan teknologi informasi, pengolahan data yang efisien menjadi hal yang sangat penting dalam dunia pemrograman. Salah satu cara untuk mengelola data dengan baik adalah dengan menggunakan struktur data yang sesuai. Struktur data berperan dalam mengatur bagaimana data disimpan, diakses, dan dimanipulasi agar program dapat berjalan lebih optimal.

Salah satu struktur data yang sering digunakan adalah Queue, yaitu struktur data yang bekerja dengan prinsip FIFO (First In First Out), di mana data yang pertama masuk akan menjadi data pertama yang keluar. Konsep ini banyak digunakan dalam berbagai aplikasi, seperti sistem antrian, pengelolaan proses, penjadwalan tugas, serta pengiriman data dalam sistem komputer.

Oleh karena itu, melalui praktikum ini, penulis mempelajari konsep dasar Queue serta implementasinya menggunakan bahasa pemrograman Java, sehingga diharapkan dapat memahami cara kerja dan penerapannya dalam pengolahan data.

1.2 TUJUAN PRAKTIKUM

1. Memahami konsep dasar struktur data Queue
2. Mengetahui prinsip kerja FIFO (First In First Out)
3. Mampu mengimplementasikan Queue menggunakan bahasa pemrograman Java
4. Memahami operasi dasar Queue seperti enqueue dan dequeue

1.3 MANFAAT PRAKTIKUM

1. Menambah pemahaman tentang struktur data Queue
2. Meningkatkan kemampuan pemrograman dalam Java
3. Membantu menyelesaikan permasalahan yang berkaitan dengan antrian data

BAB II
PEMBAHASAN PRAKTIKUM

2.1 Implementasi Queue Menggunakan Array

Pertama tama kita buat dulu class QueueArray.

Class QueueArray ini adalah implementasi struktur data queue (antrian) menggunakan array dengan konsep circular queue, jadi data yang masuk akan keluar dengan prinsip FIFO dan indeksnya bisa berputar agar tidak membuang ruang kosong. Saat objek dibuat, kapasitas ditentukan dan antrian masih kosong dengan front di depan dan rear di belakang. Method isFull dan isEmpty dipakai untuk mengecek kondisi penuh atau kosong, sedangkan enqueue menambahkan data ke belakang dengan menggeser rear menggunakan modulo lalu menambah size, dan dequeue mengambil data dari depan dengan menggeser front. Method front dan rear hanya untuk melihat elemen depan dan belakang tanpa menghapus, sementara display digunakan untuk menampilkan isi antrian meskipun belum sepenuhnya benar untuk kondisi circular, sehingga secara umum kode ini sudah menggambarkan cara kerja queue berbasis array.

```java
package pekan4_2511533005;

public class QueueArray_2511533005 {
    int front_3005, rear_3005, size_3005;
    int capacity_3005;
    int array[];

    public QueueArray_2511533005(int capacity_3005) {
        this.capacity_3005= capacity_3005;
        front_3005= this.size_3005=0;
        rear_3005=capacity_3005-1;
        array = new int[this.capacity_3005];
        }

    boolean isFull(QueueArray_2511533005 queue) {
        return (queue.size_3005==queue.capacity_3005);
    }

    boolean isEmpty(QueueArray_2511533005 queue) {
        return (queue.size_3005==0);
    }

    void enqueue_3005(int item) {
        if (isFull(this))
            return;
        this.rear_3005= (this.rear_3005+1) % this.capacity_3005;
        this.array[this.rear_3005] = item;
        this.size_3005= this.size_3005+1;
        System.out.println(item + " enqueued to Queue");
    }

    int dequeue_3005() {
        if(isEmpty(this))
            return Integer.MIN_VALUE;
        int item= this.array[this.front_3005];
        this.front_3005 = (this.front_3005+1) % this.capacity_3005;
        return item;
    }
    int front_3005() {
        if(isEmpty(this))
```

```java
    }

    int dequeue_3005() {
        if(isEmpty(this))
            return Integer.MIN_VALUE;
        int item= this.array[this.front_3005];
        this.front_3005 = (this.front_3005+1) % this.capacity_3005;
        return item;
    }
    int front_3005() {
        if(isEmpty(this))
            return Integer.MIN_VALUE;

        return this.array[this.front_3005];
    }
    int rear_3005() {
        if(isEmpty(this))
            return Integer.MIN_VALUE;
        return this.array[this.rear_3005];
    }
        //mencetak elemen antrian
    void display_3005() {
        int i;
        if (front_3005==rear_3005) {
            System.out.printf("\nAntrian Kosong\n");
            return;
        }
        // kunjungi dari belakang dan cetak
        for(i= front_3005; i<rear_3005; i++) {
            System.out.printf("%d<-- ", array[i]);
        }
        return;
    }
}
```

2.2 Pengujian Queue (Class Driver)

Selanjutnya kita bikin Class baru berjudul QueueArrayDriver sebagai driver atau main program dari Class QueueArray

Class QueueArrayDriver digunakan untuk menjalankan dan menguji queue yang telah dibuat pada Class sebelumnya. Pada main method, program membuat objek queue dengan kapasitas 1000, lalu menambahkan beberapa data menggunakan enqueue yaitu 10, 20, 30, dan 40 sehingga masuk secara berurutan. Setelah itu ditampilkan elemen depan dengan front, elemen belakang dengan rear, serta seluruh isi queue menggunakan display. Kemudian dilakukan dequeue untuk menghapus satu data dari depan, lalu program kembali menampilkan elemen depan, belakang, dan isi queue untuk melihat perubahan. Secara sederhana, class ini berfungsi untuk menunjukkan cara kerja queue dari proses penambahan, penghapusan, hingga penampilan data.

```java
package pekan4_2511533005;

public class QueueArrayDriver_2511533005 {

    public static void main(String[] args) {
        QueueArray_2511533005 queue= new QueueArray_2511533005(1000);
        queue.enqueue_3005(10);
        queue.enqueue_3005(20);
        queue.enqueue_3005(30);
        queue.enqueue_3005(40);
        System.out.println("Item di depan "+ queue.front_3005);
        System.out.println("item paling belakang " + queue.rear_3005());
        System.out.println("tampilan queue");
        queue.display_3005();
        System.out.println();
        System.out.println(queue.dequeue_3005() + " dihapus dari queue ");
        System.out.println("item didepan: " +queue.front_3005());
        System.out.println("item dibelakang: " +queue.rear_3005());
        System.out.println("tampilan queue setelah satu data dihapus");
        queue.display_3005();
    }

}
```

2.3 Reverse Data pada Queue Menggunakan Stack

Lalu kita bikin Class baru lagi yang berjudul ReverseData

Class ReverseData digunakan untuk membalik urutan data dalam queue dengan bantuan stack. Pada method main, pertama dibuat queue menggunakan LinkedList lalu diisi data 1, 2, dan 3 sehingga urutannya masih normal. Setelah itu dibuat sebuah stack untuk membantu proses pembalikan. Program kemudian memindahkan semua elemen dari queue ke stack menggunakan perulangan, sehingga urutan data otomatis terbalik karena sifat stack yang LIFO. Selanjutnya, data dari stack dimasukkan kembali ke queue, sehingga urutan dalam queue menjadi terbalik dari semula. Terakhir, hasil sebelum dan sesudah reverse ditampilkan untuk menunjukkan bahwa data berhasil dibalik dari yang awalnya [1, 2, 3] menjadi [3, 2, 1].

```java
package pekan4_2511533005;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;
public class ReverseData_2511533005 {
    public static void main(String[] args) {
        Queue<Integer> q_3005= new LinkedList<Integer>();
        q_3005.add(1);
        q_3005.add(2);
        q_3005.add(3);
        System.out.println("Sebelum reverse " + q_3005);
        Stack<Integer> s_3005 = new Stack<Integer>();
        while (!q_3005.isEmpty()) { // Q->S
            s_3005.push(q_3005.remove());
        }
        while (!s_3005.isEmpty()) { // S->Q
            q_3005.add(s_3005.pop());
        }
        System.out.println("Sesudah reverse = " +q_3005); //[3, 2, 1]
    }

}
```

2.4 Iterasi Queue Menggunakan Iterator

Kita bikin lagi Class baru yang berjudul IterasiQueue

Class IterasiQueue digunakan untuk menampilkan isi queue dengan cara melakukan iterasi menggunakan Iterator. Pada main method, dibuat sebuah queue bertipe String menggunakan LinkedList, kemudian ditambahkan beberapa data yaitu "Praktikum", "Struktur", "Data", "Dan", dan "Algoritma" secara berurutan. Setelah itu dibuat objek Iterator untuk mengakses elemen dalam queue, lalu dilakukan perulangan menggunakan while dengan kondisi hasNext untuk mengecek apakah masih ada elemen berikutnya. Setiap elemen yang ada di dalam queue kemudian ditampilkan satu per satu menggunakan next. Secara sederhana, class ini menunjukkan bagaimana cara menelusuri dan menampilkan isi queue tanpa menghapus datanya.

```java
package pekan4_2511533005;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;
public class IterasiQueue_2511533005 {
    public static void main(String[] args) {
        {
            Queue<String> q_3005= new LinkedList<>();

            q_3005.add("Praktikum");
            q_3005.add("Struktur");
            q_3005.add("Data");
            q_3005.add("Dan");
            q_3005.add("Algoritma");
            Iterator<String> iterator = q_3005.iterator();
            while (iterator.hasNext()) {
                System.out.print(iterator.next() + " ");
            }
        }

    }

}
```

2.5 Implementasi Queue Menggunakan LinkedList

Lalu kita buat class terakhir di praktikum yaitu Class QueueLinkedList.

Class QueueLinkedList merupakan contoh penggunaan queue dengan memanfaatkan LinkedList dari Java sebagai implementasinya. Pada main method, pertama dibuat sebuah queue bertipe Integer, kemudian ditambahkan elemen dari 0 sampai 5 menggunakan perulangan sehingga data masuk secara berurutan. Setelah itu, isi antrian ditampilkan untuk melihat data yang ada. Selanjutnya dilakukan penghapusan elemen terdepan menggunakan remove, lalu hasil antrian ditampilkan kembali untuk melihat perubahan. Program juga menggunakan peek untuk melihat elemen paling depan tanpa menghapusnya, serta size untuk mengetahui jumlah elemen dalam antrian. Secara sederhana, class ini menunjukkan operasi dasar queue seperti menambah, menghapus, melihat elemen depan, dan menghitung jumlah data dengan menggunakan struktur bawaan Java.

```java
package pekan4_2511533005;
import java.util.LinkedList;
import java.util.Queue;
public class QueueLinkedList_2511533005 {
        public static void main(String[] args) {
            Queue<Integer> q_3005 = new LinkedList<>();
            // tambah elemen {0, 1, 2, 3, 4,5} ke antrian
            for (int i = 0; i < 6; i++)
                q_3005.add(i);
            // Menampilkan isi antrian.
            System.out.println("Elemen Antrian " + q_3005);
            // Untuk menghapus kepala antrian.
            int hapus = q_3005.remove();
            System.out.println("Hapus elemen = " + hapus);
            System.out.println(q_3005);
            // Untuk melihat antrian terdepan
            int depan = q_3005.peek();
            System.out.println("Kepala Antrian = " + depan);

            int banyak = q_3005.size();
            System.out.println("Size Antrian = " + banyak);
        }
    }
```

BAB III
KESIMPULAN

3.1 KESIMPULAN

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Queue merupakan salah satu metode pengelolaan data yang bekerja dengan prinsip FIFO (First In First Out), di mana data yang pertama masuk akan menjadi data pertama yang keluar. Melalui beberapa implementasi yang telah dibuat, seperti queue menggunakan array, penggunaan class driver, pembalikan data dengan bantuan stack, iterasi menggunakan iterator, serta implementasi menggunakan LinkedList, dapat dipahami bahwa setiap metode memiliki cara kerja yang berbeda namun tetap mengacu pada konsep dasar queue. Praktikum ini juga menunjukkan bahwa penggunaan struktur data yang tepat dapat mempermudah pengolahan data dalam pemrograman, khususnya dalam bahasa Java.

DAFTAR PUSTAKA

GeeksforGeeks. *Queue Data Structure*.
https://www.geeksforgeeks.org/queue-data-structure/

Programiz. *Queue Data Structure*.
https://www.programiz.com/dsa/queue

TutorialsPoint. *Data Structures - Queue*.
https://www.tutorialspoint.com/data_structures_algorithms/dsa_queue.htm